
COFRE / IOT

Cofre Eletrônico com Senha por Arduino

Alunos:

João Pedro Martins - RA: 22.124.034-3

Yuri Lucas Oishi - RA: 22.225.025-0

Wellington Menezes Paim - RA:22.225.024-3

Relatório Técnico

Disciplina: CCM520 – IOT
2026

1. Objetivo do Projeto

O projeto Cofre tem como objetivo projetar e implementar um sistema embarcado de controle de acesso para um compartimento seguro, simulando um cofre eletrônico com múltiplos periféricos integrados ao Arduino Uno.

O sistema permite ao usuário inserir uma senha de quatro dígitos pressionando botões físicos. Ao digitar a sequência correta, um servo motor é acionado simulando a abertura de uma trava mecânica. Erros consecutivos ativam um mecanismo de bloqueio temporário com contador regressivo exibido no LCD.

Os objetivos específicos do projeto são:

- Integrar periféricos digitais e analógicos ao Arduino Uno em um sistema coeso
- Implementar lógica de controle de acesso com verificação de senha de 4 dígitos
- Fornecer feedback multissensorial: visual (LCD e LEDs), sonoro (buzzer) e mecânico (servo)
- Implementar um mecanismo de bloqueio por tentativas com countdown no display
- Demonstrar boas práticas de programação embarcada em C/C++ para Arduino

2. Descrição Geral do Sistema

O sistema é desenvolvido em linguagem C/C++ para Arduino Uno e simula o funcionamento de um cofre eletrônico de senha. Ao ligar o dispositivo, o servo motor é posicionado a 0° (trava fechada) e o LCD exibe a tela inicial com a mensagem "LOCKER INTELIG." na linha 1 e "Digite a senha:" na linha 2.

A senha é inserida por meio de quatro botões físicos (BTN1 a BTN4), cada um representando um dígito de 1 a 4. A cada botão pressionado, um asterisco "*" aparece no LCD indicando o progresso da digitação, e um bip de confirmação é emitido pelo buzzer. Após quatro toques, o sistema compara automaticamente a sequência digitada com a senha armazenada.

Senha configurada no projeto

Sequência: BTN2 → BTN3 → BTN2 → BTN3

Representação interna: { 2, 3, 2, 3 }

A senha pode ser alterada diretamente na variável `senhaCorreta[]` no código-fonte.

Em caso de senha correta, o servo gira para 90° (cofre aberto), o LED verde acende e o buzzer emite dois bips ascendentes. O cofre permanece aberto até que o usuário pressione BTN1 para fechá-lo. Em caso de erro, o LED vermelho pisca, o buzzer emite um tom grave e o número de tentativas restantes é exibido no LCD. Após três erros consecutivos, o sistema entra em modo de bloqueio com alarme sonoro e countdown de 10 segundos.

3. Recursos de Hardware

3.1 Microcontrolador

O projeto utiliza o Arduino Uno como unidade central de processamento. Ele é baseado no microcontrolador ATmega328P com clock de 16 MHz, 32 KB de memória Flash, 2 KB de SRAM e 1 KB de EEPROM. Conta com 14 pinos digitais (6 com suporte a PWM) e 6 pinos analógicos, sendo suficiente para todos os periféricos do projeto.

3.2 Lista de Componentes e Pinos

Componente	Pino(s)	Função no Projeto
LCD	A0–A5	Interface visual — exibe mensagens, asteriscos e countdown
Servo Motor SG90	D9 (PWM)	Simula trava mecânica: 0° = fechado, 90° = aberto
BTN1	D2	Dígito '1' da senha — também fecha o cofre após abertura
BTN2	D3	Dígito '2' da senha
BTN3	D4	Dígito '3' da senha
BTN4	D5	Dígito '4' da senha
LED Verde	D6	Acende quando o cofre está aberto (acesso autorizado)
LED Vermelho	D7	Acende em standby, erro, bloqueio
Buzzer Passivo	D8	Feedback sonoro: confirmação, erro, alarme de bloqueio
Resistores 330 Ω	—	Limitação de corrente para os LEDs verde e vermelho
Protoboard + fios	—	Montagem e conexões entre os componentes

3.3 Justificativa dos Componentes

LCD

O display LCD 16x2 foi escolhido por oferecer interface visual clara e de baixo custo. O modo de operação de 4 bits (usando apenas D4–D7 além de RS e EN) reduz o consumo de pinos do Arduino de 8 para 6, sendo todos pinos analógicos usados como digitais (A0–A5). A biblioteca LiquidCrystal gerencia internamente o protocolo de comunicação paralela.

Quatro Botões com INPUT_PULLUP

Os quatro botões utilizam o resistor pull-up interno do ATmega328P, ativado pela diretiva INPUT_PULLUP. Isso elimina a necessidade de resistores externos e garante que o pino leia HIGH quando o botão está solto e LOW quando pressionado — lógica invertida, verificada com !digitalRead(). O debounce é implementado por delay(300 ms) após cada registro.

Servo Motor

O servo motor SG90 é controlado por sinal PWM no pino D9. A biblioteca Servo.h abstrai a geração do sinal, permitindo posicionamento angular direto com servo.write(ângulo). O ângulo 0° representa o cofre fechado e 90° representa o cofre aberto, simulando o movimento de uma trava mecânica unilateral.

LEDs Indicadores

Dois LEDs fornecem feedback visual imediato do estado do sistema: verde para acesso autorizado e vermelho para standby, erro ou bloqueio. Resistores de 330 Ω em série limitam a corrente a aproximadamente 9 mA com alimentação de 5 V, dentro dos limites do pino digital do Arduino (máx. 40 mA).

Buzzer Passivo

O buzzer passivo permite gerar diferentes frequências de tom usando a função `tone()`, tornando possível diferenciar os eventos sonoramente: 1000 Hz para confirmação de toque, 300 Hz (tom grave) para senha errada, 500 Hz (bips repetidos) para alarme de bloqueio, e 1500/2000 Hz (bips ascendentes) para abertura bem-sucedida.

4. Recursos de Software e Bibliotecas

4.1 Bibliotecas Utilizadas

```
código
#include <LiquidCrystal.h> // Controle do display LCD 16x2
#include <Servo.h>         // Controle do servo motor por PWM
```

Ambas as bibliotecas são nativas da IDE do Arduino e não requerem instalação adicional.

4.2 Biblioteca `LiquidCrystal.h`

Gerencia a comunicação paralela com o display LCD HD44780 em modo 4 bits. A instância é criada informando os pinos RS, EN, D4, D5, D6 e D7 no construtor:

```
código
LiquidCrystal lcd(A0, A1, A2, A3, A4, A5);
```

As principais funções utilizadas no projeto são `lcd.begin(16, 2)` para inicialização, `lcd.clear()` para limpar o display, `lcd.setCursor(col, row)` para posicionamento do cursor e `lcd.print()` para exibição de texto e números.

4.3 Biblioteca `Servo.h`

Controla servos por sinal PWM (frequência de 50 Hz, pulso de 1–2 ms). A instância `Servo` é associada ao pino com `servo.attach(SERVO_PIN)` e posicionada com `servo.write(ângulo)`. No projeto, os dois ângulos utilizados são:

```
código
servo.write(0); // Cofre fechado (posição de repouso da trava)
servo.write(90); // Cofre aberto (90° em relação à posição inicial)
```

4.4 Funções Arduino Utilizadas

Função	Uso no Projeto
<code>pinMode()</code>	Configura botões como <code>INPUT_PULLUP</code> e saídas (LEDs, buzzer)
<code>digitalRead()</code>	Lê o estado dos 4 botões no loop principal
<code>digitalWrite()</code>	Liga/desliga os LEDs verde e vermelho
<code>tone(pino, freq, dur)</code>	Aciona o buzzer com frequência e duração específicas
<code>servo.attach()</code>	Associa o objeto <code>Servo</code> ao pino PWM D9
<code>servo.write(ângulo)</code>	Posiciona o servo em 0° (fechado) ou 90° (aberto)

delay(ms)	Debounce dos botões (300 ms) e pausas entre mensagens
lcd.begin()	Inicializa o LCD com 16 colunas e 2 linhas
lcd.clear()	Limpa o conteúdo do display antes de nova mensagem
lcd.setCursor()	Posiciona o cursor para escrita precisa no LCD
lcd.print()	Exibe texto e valores numéricos no display

5. Funcionalidades Implementadas

5.1 Tela Inicial e Estado de Standby

A função `telaInicial()` é chamada ao iniciar o sistema e ao término de cada ciclo (após abertura, erro ou desbloqueio). Ela posiciona o servo em 0°, acende o LED vermelho como indicador de standby e exibe:

```
código
lcd.print("LOCKER INTELIG."); // Linha 1: nome do sistema
lcd.print("Digite a senha:"); // Linha 2: instrução ao usuário
```

5.2 Registro de Dígitos por Botões

O loop principal monitora continuamente os quatro botões usando lógica invertida (`INPUT_PULLUP`). Ao detectar um botão pressionado, chama `registrar(n)`, onde `n` é o número do botão (1 a 4):

```
código
void loop() {
  if (!digitalRead(BTN1)) { registrar(1); delay(300); }
  if (!digitalRead(BTN2)) { registrar(2); delay(300); }
  if (!digitalRead(BTN3)) { registrar(3); delay(300); }
  if (!digitalRead(BTN4)) { registrar(4); delay(300); }
}
```

A função `registrar()` armazena o dígito no vetor `tentativa[]`, incrementa o índice `posicao` e atualiza o LCD com asteriscos. Ao completar 4 dígitos, chama `verificar()` automaticamente.

5.3 Feedback Visual no LCD

O display LCD 16x2 orienta o usuário em todas as etapas do sistema. A tabela abaixo resume todas as mensagens exibidas:

Situação	Mensagem no LCD
Sistema ligado / reset	"LOCKER INTELIG." / "Digite a senha:"
Digitando a senha	"LOCKER INTELIG." / "***" (asteriscos progressivos)
Senha correta	" COFRE ABERTO!" / "B1 = fechar"
Senha incorreta	" SENHA ERRADA!" / "Restam: X"
Sistema bloqueado	"!! BLOQUEADO !!" / "Aguarde: Xs" (countdown)

5.4 Verificação da Senha

Ao completar os 4 dígitos, a função `verificar()` compara o vetor `tentativa[]` com `senhaCorreta[]` posição a posição. A comparação encerra na primeira divergência (`break`) para eficiência:

```
código
void verificar() {
    bool certa = true;
    for (int i = 0; i < 4; i++) {
        if (tentativa[i] != senhaCorreta[i]) {
            certa = false;
            break;
        }
    }
    posicao = 0; // reseta índice para próxima tentativa
    if (certa) abrirCofre();
    else { erros++; if (erros >= 3) bloquear(); else senhaErrada(); }
}
```

5.5 Abertura e Fechamento do Cofre

Quando a senha está correta, a função `abrirCofre()` executa a seguinte sequência:

1. Gira o servo para 90° — trava aberta
2. Acende o LED verde e apaga o LED vermelho
3. Emite dois bips ascendentes (1500 Hz e 2000 Hz) como som de vitória
4. Exibe "COFRE ABERTO!" e "B1 = fechar" no LCD
5. Entra em loop de espera até o usuário pressionar BTN1
6. Retorna o servo a 0°, apaga o LED verde, acende o vermelho e emite bip de confirmação
7. Zera o contador de erros e volta à tela inicial

```
código
void abrirCofre() {
    servo.write(90);
    digitalWrite(LED_VERDE, HIGH);
    digitalWrite(LED_VERMELHO, LOW);
    tone(BUZZER, 1500, 150); delay(200); tone(BUZZER, 2000, 150);
    lcd.print(" COFRE ABERTO!");
    lcd.print("B1 = fechar");
    while (digitalRead(BTN1)) { } // aguarda botão de fechamento
    servo.write(0);
    erros = 0;
    telaInicial();
}
```

5.6 Tratamento de Senha Incorreta

A função `senhaErrada()` fornece feedback negativo ao usuário: pisca o LED vermelho, emite um tom grave (300 Hz por 400 ms) e exibe a mensagem de erro com o número de tentativas restantes calculado como (3 – erros). A mensagem fica visível por 1,5 segundo antes de retornar à tela inicial.

```
código
void senhaErrada() {
    digitalWrite(LED_VERMELHO, HIGH);
    tone(BUZZER, 300, 400);
    delay(400);
    digitalWrite(LED_VERMELHO, LOW);
    lcd.print(" SENHA ERRADA!");
}
```

```

    lcd.print("Restam: "); lcd.print(3 - erros);
    delay(1500);
    telaInicial();
}

```

5.7 Bloqueio Temporário por Tentativas Excedidas

Após o terceiro erro consecutivo, a função bloquear() é acionada. Ela emite um alarme composto por 6 bips de 500 Hz e exibe um countdown de 10 segundos no LCD, atualizado a cada segundo por delay(1000). Ao término do bloqueio, o contador de erros é zerado e o sistema retorna ao estado normal.

```

código
void bloquear() {
    digitalWrite(LED_VERMELHO, HIGH);
    for (int i = 0; i < 6; i++) { tone(BUZZER, 500, 200); delay(300); }
    for (int t = 10; t > 0; t--) {
        lcd.print("!! BLOQUEADO !!");
        lcd.print("Aguarde: "); lcd.print(t); lcd.print("s");
        delay(1000);
    }
    erros = 0;
    digitalWrite(LED_VERMELHO, LOW);
    telaInicial();
}

```

Resumo dos Feedbacks Sonoros

1000 Hz, 80 ms	→ Confirmação de toque em botão
300 Hz, 400 ms	→ Senha incorreta (tom grave)
500 Hz, 200 ms × 6	→ Alarme de bloqueio
1500 Hz + 2000 Hz	→ Abertura bem-sucedida (bips ascendentes)
800 Hz, 200 ms	→ Confirmação de fechamento do cofre

6. Estrutura do Código-Fonte

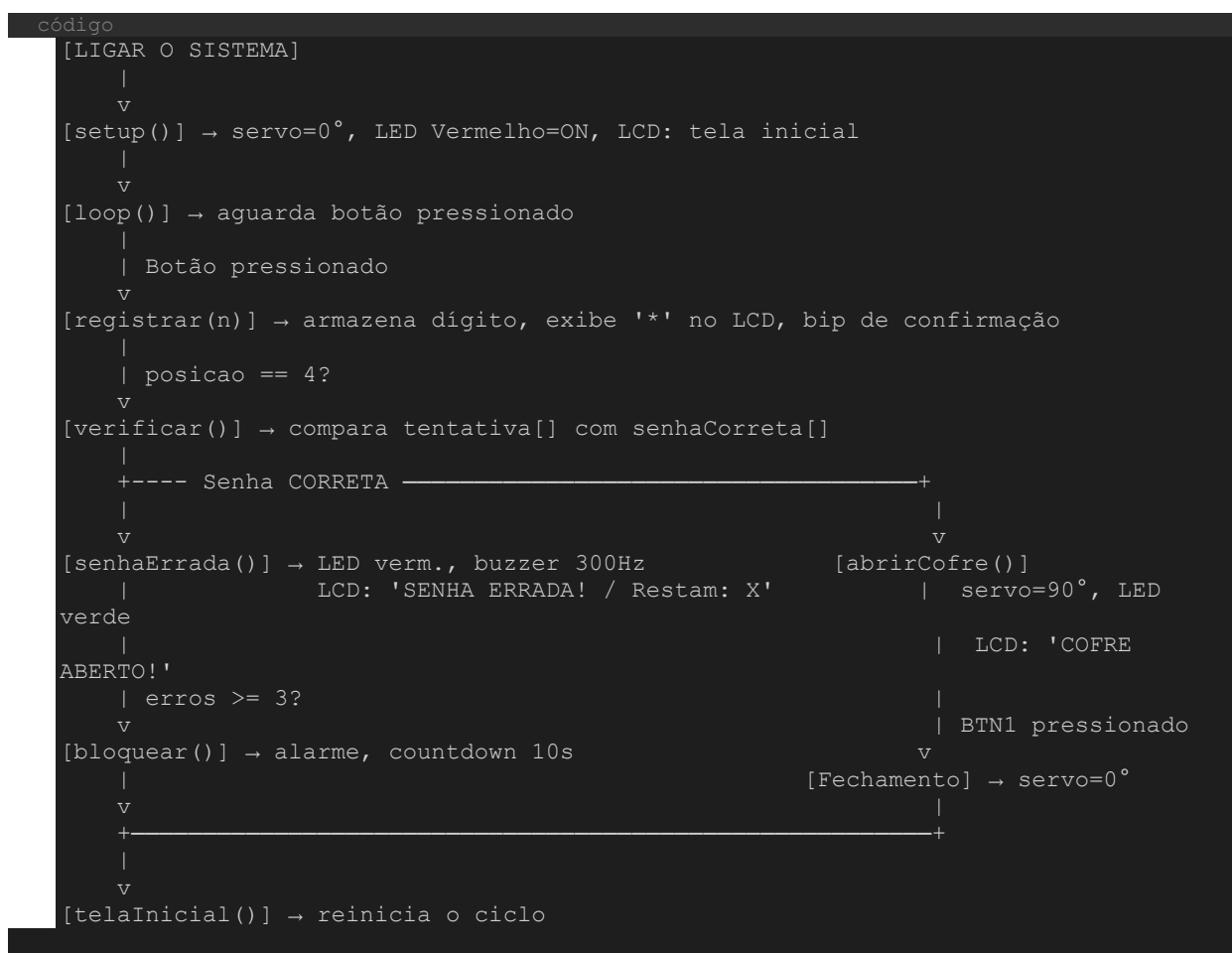
O código está organizado em um único arquivo .ino com as seguintes funções:

Função	Responsabilidade
setup()	Configura pinos, inicializa servo (0°), LCD (16×2) e chama telaInicial()
loop()	Monitora os 4 botões continuamente com lógica INPUT_PULLUP invertida
registrar(int btn)	Armazena o dígito, atualiza asteriscos no LCD, chama verificar() ao 4º dígito
verificar()	Compara tentativa[] com senhaCorreta[], decide entre abrirCofre(), senhaErrada() ou bloquear()
abrirCofre()	Aciona servo 90°, LEDs, buzzer, aguarda BTN1 para fechar

senhaErrada()	Feedback negativo com LED, buzzer e contagem de tentativas restantes no LCD
bloquear()	Alarme sonoro, countdown de 10 s no LCD, zera erros ao final
telaInicial()	Exibe tela de standby, coloca LED vermelho e garante servo em 0°

7. Fluxo de Operação do Sistema

O fluxo de funcionamento do sistema é apresentado abaixo de forma textual. Cada etapa corresponde a uma função ou transição de estado no código:



8. Testes Realizados

Os testes foram conduzidos de forma funcional, verificando cada cenário possível de interação com o sistema:

Caso de Teste	Ação Realizada	Resultado Esperado	Resultado Obtido
Inicialização	Ligar o Arduino	LCD: tela inicial; LED verm. aceso; servo em 0°	Funcionou
Registro de dígito	Pressionar BTN2	Asterisco no LCD; bip de confirmação	Funcionou
4 dígitos corretos	Digitar 2→3→2→3	Cofre abre; LED verde; bips ascendentes	Funcionou
Fechamento do cofre	Pressionar BTN1 após abertura	Servo volta a 0°; LED vermelho; bip de fechamento	Funcionou
Senha incorreta (1ª)	Digitar sequência errada	LED verm. pisca; buzzer 300 Hz; 'Restam: 2'	Funcionou
Senha incorreta (2ª)	Digitar errado novamente	LED verm.; buzzer; 'Restam: 1'	Funcionou
3º erro → bloqueio	Digitar errado 3ª vez	Alarme, countdown 10→1 no LCD, desbloqueio auto.	Funcionou
Desbloqueio automático	Aguardar 10 segundos	Sistema volta à tela inicial normalmente	Funcionou
Reset de erros	Acertar a senha após erros	Contador de erros zerado ao fechar cofre	Funcionou

9. Limitações e Melhorias Sugeridas

O projeto, apesar de funcional e didaticamente completo, apresenta algumas limitações técnicas que podem ser endereçadas em versões futuras:

Limitação Identificada	Melhoria Sugerida
Senha fixa no código-fonte	Armazenar a senha na EEPROM e permitir alteração pelo usuário via modo de programação
Debounce por delay() bloqueante	Substituir por debounce com millis() para não bloquear o loop e melhorar responsividade
Loop bloqueante na abertura (while)	Implementar máquina de estados explícita com enum Estado, eliminando loops de espera
Bloqueio temporário (10 s)	Persistir bloqueio na EEPROM para resistir a desligamento durante o período de bloqueio
Senha exposta em memória Flash	Armazenar hash da senha (ex: CRC8 ou SHA) em vez do valor em claro

Sem registro de eventos	Adicionar módulo RTC DS3231 + cartão SD para log com data/hora de tentativas e aberturas
Ausência de sensor de violação	Adicionar reed switch ou sensor LDR para detectar abertura física indevida do cofre
Sem comunicação externa	Integrar módulo ESP8266 ou HC-05 para notificações remotas por Wi-Fi ou Bluetooth

10. Conclusão

O projeto Locker Inteligente demonstrou com sucesso a integração de múltiplos periféricos em torno do microcontrolador Arduino Uno para implementar um sistema funcional de controle de acesso por senha. Todos os requisitos fundamentais foram cumpridos: entrada de senha por botões, feedback visual no LCD, feedback sonoro diferenciado por frequência, acionamento do servo motor como trava, contagem de tentativas erradas e bloqueio temporário com countdown.

A experiência permitiu consolidar conhecimentos práticos de programação em C/C++ para sistemas embarcados, incluindo o uso de bibliotecas nativas (LiquidCrystal e Servo), gerenciamento de pinos digitais com INPUT_PULLUP, geração de tons por PWM e controle de atuadores angulares.

As melhorias identificadas — como uso de máquina de estados formal com enum, persistência em EEPROM, debounce não-bloqueante com millis() e integração de comunicação sem fio — apontam um caminho claro para evolução do projeto em direção a um produto embarcado mais robusto e seguro, compatível com os requisitos de um dispositivo de Internet das Coisas (IoT).